COMP364 Quiz 2

Question 1. Pros and cons of nearest neighbor (2 points)

List the advantages and disadvantages of the nearest neighbor classifier.

Question 2. Pros and cons of large k in nearest neighbor (2 points)

List the advantages and disadvantages of using a large value of k for nearest neighbor classification.

Question 3. Definition of expectation (2 points)

Let X be a discrete real-valued random variable whose possible values are $x_1, x_2, \ldots, x_n$, with corresponding probabilities $p(x_1), p(x_2), \ldots, p(x_n)$. Give a formula for $E(X)$, the expectation of X.

Question 4. Definition of entropy (2 points)

Let X be as in the previous question. Give a formula for $H(X)$, the entropy of X.

Question 5. Computing entropy (15 points)

Compute the entropy of the following distribution:

x	apple	banana	lemon	grape	plum
$p(x)$	$\frac{1}{16}$	$\frac{1}{2}$	$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{4}$

We now introduce a data set termed the *widget* data set. Here are the widget data set's attributes and corresponding values:

$$\begin{aligned} \text{Color} &= \{\text{Blue}, \text{Green}, \text{Yellow}\} \\ \text{Sound} &= \{\text{Quiet}, \text{Loud}\} \\ \text{Texture} &= \{\text{Rough}, \text{Smooth}\} \\ \text{Material} &= \{\text{Wood}, \text{Metal}, \text{Fiberglass}\} \end{aligned}$$

The class variable is *Material*. Below is the widget data set itself, containing eight instances. Each

attribute value is abbreviated to its first letter.

	Color	Sound	Texture	Material
1	B	Q	R	W
2	B	L	R	W
3	B	Q	S	M
4	B	L	S	M
5	G	Q	S	W
6	G	L	S	W
7	Y	Q	R	F
8	Y	L	R	F

Question 6. Computing first decision tree split attribute (15 points)

Compute the first split attribute for the widget data set.

Question 7. Computing next decision tree split attribute (15 points)

Compute the split attribute for the Blue node of the above tree.

Question 8. Assembling complete decision tree (5 points)

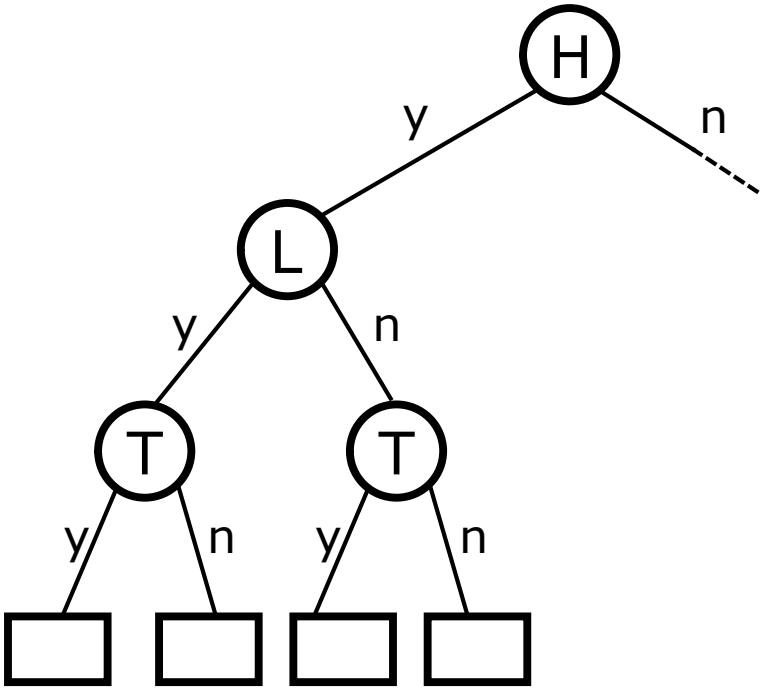
Draw the complete decision tree for the widget example above.

Question 9. Working with a more complex decision tree

The computer science students at Red Devil College are worried about passing the notoriously-difficult AI course, and they decide to build a decision tree to determine whether students are likely to pass or fail the course. The current students collect data from several previous years' students, based on answers to three yes/no questions: (i) Did you complete all assigned readings from the *textbook*? (ii) Did you attend all of the *lectures*? (iii) Did you complete all of the *homework* assignments? These questions are abbreviated as T for textbook, L for lectures, and H for homework; answers to the questions are abbreviated as y for "yes" and n for "no". The previous years' students were also labeled according to whether they passed (P) or failed (F) the course. The data collected from 1789 students is summarized in the following table:

				Homework (H)			
				y		n	
				grade		grade	
				P	F	P	F
Textbook (T)	y	Lectures (L)	y	402	0	350	31
			n	390	10	150	72
	n	Lectures (L)	y	220	5	35	40
			n	63	23	0	23

The figure below shows a partially-completed decision tree constructed from the above data. You may assume that the tree has been constructed correctly according to the algorithm studied in class.



(a) **(4 points)** Fill in each of the leaves in the partially-completed tree above with the correct decision (i.e. P or F.)

(b) **(2 points)** Which nodes in the partially-completed tree above are pure nodes?

(c) **(15 points)** Complete the tree using the algorithm studied in class. Include all labels for edges and nodes. For full credit, you must explain your reasoning. To speed your calculation, you may use without proof the fact that, when we take the instances for which $H =$ "no" and split them on the attribute L, the expected entropy of the resulting split is 0.743.

Question 10. PyTorch matrix addition and broadcasting (12 points)

Assume that `import torch` has already been executed. For each part, write the output produced by the final `print` statement.

(a)

```
A = torch.tensor([
    [1, 2],
    [3, 4],
    [5, 6],
])
B = torch.tensor([
    [10, 20],
    [30, 40],
    [50, 60],
])
C = A + B
print(C)
```

(b)

```
A = torch.tensor([
    [1, 2],
    [3, 4],
    [5, 6],
])
v = torch.tensor([
    [10],
    [20],
    [30],
])
C = A + v
print(C)
```

(c)

```
v = torch.tensor([
    [1],
    [2],
    [3],
])
w = torch.tensor([[10, 20]])
C = v + w
print(C)
```

Total number of points: **91**